IST688 Setup — Read This First

You got five files: this README, `CLAUDE.md`, `app.py`, `pyproject.toml`, and `requirements.txt`.

The easiest way to do this setup: open claude.ai, start a new chat, attach all five files, and say:

> Walk me through the README one gate at a time. I'm on [Mac / Windows].

Claude will take it from there. Or follow the steps below yourself. Either works.

The rule: every gate ends with a **pass signal** — something you can see on your screen. Don't move on until you see it. If something breaks, copy the *exact* error text and paste it to Claude, or into the course channel. Not a description of it. The text.

Timing matters. Gate 1 takes an afternoon, mostly waiting: about an hour for SU to turn on Claude, a 20-minute training video, then about another hour for Claude Code. So: **start Gate 1 today, at least a day before you plan to do the rest.** The rest takes about 45 minutes. Do all of it before the first lab.

The gate format and the first few gates are adapted, with thanks, from Mike Fudge's IST300 setup checklist.

The rule that saves you an hour

Install something → quit Claude completely → reopen it.

Claude only notices new software when it starts up. If you install something while Claude is open, Claude will insist it isn't installed. It is. Quit and reopen. Same for terminal windows — after installing a command-line tool, close the old terminal and open a new one.

Gate 1 — Claude account, through SU

Syracuse gives you Claude for free, but there are two separate things to request, in order: **Claude Chat** first, then **Claude Code**. Do **not** sign up at claude.com with a personal email — a personal free account won't have the Code tab and you'll have to redo this.

Part A — Request Claude Chat (do this today)

1. Go to https://getclaude.syr.edu. You'll see three cards: Claude Chat, Claude Cowork, Claude Code.
2. On the **Claude Chat** card, click **Request Claude Chat**. Sign in with your `NetID@syr.edu` if asked.
3. In the portal that opens, click **Continue**, choose **I Agree** on the usage policy, click **Submit request**. No business justification needed. Ignore the email Microsoft sends.
4. Back on getclaude.syr.edu, the Claude Chat card now says **PROVISIONING** and shows a message like *"Your account is now being added to Claude. It should be ready by [time]."* That's the normal in-progress state. Note the time it gives you.
5. **Now wait.** It's usually about an hour, but it varies. Go do something else. (Gates 2, 4, 5, and 6 don't need your Claude account — you can do those while you wait if you want.)
   - When you come back, click **Refresh My Access** on the card to check. If you see a banner saying your sign-in session expired, click **Sign in again** — that's harmless, it just re-checks.

Part B — Sign in, then request Claude Code

6. **If you already have a personal Claude account, log out of it first.** Then go to https://claude.ai, enter your `NetID@syr.edu`, click **Continue with SSO**, and finish the SU login. If it doesn't work yet, wait longer — it's not you.
7. Once you're in, go back to https://getclaude.syr.edu. The Claude Chat card should now say **ACTIVE**, and the other two cards should say **TRAINING REQUIRED**.

8. On the **Claude Code** card — the one on the **right**, not Claude Cowork in the middle; they look almost identical — click **Start training video**. It's about **20 minutes**. Watch it to the end, then confirm you've reviewed it. The card will then let you request access. Do that.
9. **Wait again** — about another hour before Claude Code turns on. Same as before: it's not you. Come back and check the card, or just move on to Gate 2 and check for the Code tab there.

You want Claude Code, not Claude Cowork. Cowork is a different tool with its own training video, and it isn't used in this course. If you sit through the wrong 20-minute video, you'll have to do the right one too.

Pass signal: you're signed in at https://claude.ai, **"Syracuse University"** appears at the top of the page, and the Claude Code card on getclaude.syr.edu says **ACTIVE**.

Gate 2 — Claude Desktop installed

The browser version can't see your files. You need the app.

1. Go to https://claude.com/download and get the version for your computer.
   - **Mac:** open the downloaded file, drag Claude into Applications.
   - **Windows:** run the installer. If your laptop is a Surface or says "Snapdragon," choose the ARM64 version.
2. Open Claude and sign in the same way as Gate 1 — your `NetID@syr.edu`, then **Continue with SSO**.
3. Make sure you can start a normal Claude conversation and that you can access the **Code** tab.

Pass signal: Claude Chat works and you can open the **Code** tab.

Cowork may look different on your version. It may appear as a separate tab, as an option inside Chat, or not as a top-level tab at all. That's fine — **we do not use Cowork in this course.**

- **No Code tab?** Either you're on a personal account (sign out fully, sign in with SSO), or your Claude Code request from Gate 1 Part B hasn't gone through yet. Check both.
- **University laptop won't let you install?** Stop and post in the course channel. There's a workaround, but it's different from this checklist.

Gate 3 — Course folder, with the files in it

You'll build more than one app this semester, so each app gets its own folder inside a course folder.

1. In **Documents**, make a new folder called `ist688`.
2. Inside `ist688`, make another folder called `app1`.
   - Keep all of this in Documents — **not** on the Desktop, and **not** inside iCloud, OneDrive, or Google Drive. Cloud sync and programming tools fight, and you lose.
3. Move the five files you downloaded — `README.md`, `CLAUDE.md`, `app.py`, `pyproject.toml`, `requirements.txt` — into `app1`. They're probably in your Downloads folder right now. Drag them over.
4. Open Claude Desktop and click the **Code** tab.
5. Open `Documents/ist688/app1` as the project. If your computer asks whether Claude can access Documents or this folder, **allow it**.

Pass signal: Claude Code opens the project and can see all five files: `README.md`, `CLAUDE.md`, `app.py`, `pyproject.toml`, and `requirements.txt`.

If it shows a different folder or different files, you pointed Claude Code at the wrong project. Better to catch that now.

Gate 4 — GitHub account

1. If you don't have one, sign up at https://github.com. Pick a username you'd put on a résumé.
2. Add your `@syr.edu` email in settings and verify it. That unlocks the free GitHub Student Developer Pack, which is worth having.

Pass signal: signed in at github.com, email shows as verified.

Gate 5 — Git installed

Windows: Download Git from https://git-scm.com/download/win and run it. Click **Next** through every screen — the defaults are fine. **Then quit Claude and reopen it.**

Mac: Open Terminal (press ⌘-space, type `terminal`, press Enter). Type `git --version` and press Enter. If it says Git isn't installed, your Mac may offer to install "command line developer tools." Say yes and wait — it can take a few minutes.

Mac: no installer appeared? Run:

```
xcode-select --install
```

If you still don't see anything, wait a couple of minutes and try `git --version` again anyway — the installation may be proceeding without an obvious popup. If it still isn't working, check **System Settings → General → Software Update** for **Command Line Tools for Xcode**.

Then, in Terminal (Mac) or PowerShell (Windows), type these two lines, one at a time, with your real name and SU email:

```
git config --global user.name "Your Name"
git config --global user.email "you@syr.edu"
```

Pass signal: `git --version` prints a version number.

Gate 6 — uv installed

uv is the tool that runs Python for you. **Don't install Python separately** — uv handles it.

Mac — in Terminal, paste this and press Enter:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows — in PowerShell, paste this and press Enter:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Close that terminal window completely. Open a brand-new Terminal or PowerShell window. Then type:

```
uv --version
```

Pass signal: a version number prints.

Now quit Claude and reopen it. Yes, again. Forgetting this step here costs the most time of anything on this list.

Gate 7 — Your app runs, and you can edit it

This is the real gate. Everything before it was setup. This is what you'll do every week.

The supplied `app.py` is the **Tip Calculator** from your getting-started materials. You have already seen what the app does at a high level; now you are going to run the actual Python code and change it yourself.

You never install Streamlit yourself. The first time you run the app, `uv` reads `pyproject.toml`, quietly installs Python and Streamlit into a hidden folder inside `app1`, and then runs your app. That first run takes a minute or two and prints a lot of text. That's normal. Every run after that is instant.

1. Open Claude Desktop, click the **Code** tab.
2. Open your `Documents/ist688/app1` folder as the project.
3. Claude Code should automatically read the project's `CLAUDE.md` instructions. If Claude ever suggests changing the project setup in a way that seems inconsistent with this checklist, tell it: **"Read CLAUDE.md and README.md first."**
4. Paste this to Claude:

   > Run the Streamlit app with `uv run streamlit run app.py --server.port 8501 --server.headless true` and open it in the preview. Save that as the project's launch configuration so it works next time with one click.

5. Try the app. Enter **$100** for the bill and choose **Good** service. The tip should be **$20.00**.

Look at and edit the Python code

6. Ask Claude Code:

   > Can I see and edit `app.py`?

7. Open `app.py` in the editor. Find the line that sets the tip for **Good** service to `0.20` and change it yourself to `0.18`.
8. Save the file. Rerun or refresh the preview if needed.
9. Enter **$100** and choose **Good** again. The tip should now be **$18.00**.

Pass signal: the Tip Calculator runs in the local preview, you can see and edit `app.py`, and your change from 20% to 18% appears in the running app.

If it doesn't work:

- **"uv: command not found"** → you didn't restart Claude after Gate 6. Quit and reopen.
- **Preview is blank and just spins** → the run command is missing `--server.headless true`. Paste the command from step 4 again, exactly.
- **Your code changed but the app didn't** → save `app.py`, then rerun or refresh the preview.
- **"Port 8501 is already in use"** → an old copy is still running. Ask Claude to stop it and try again.
- **Anything else** → copy the error, go to the **Chat** tab (or claude.ai), paste it along with this README, and ask what's wrong.

Gate 8 — Your app is live on the internet

1. Go to https://github.com and click **New repository**. Name it `ist688-app1`. Leave everything else alone — **don't** check "Add a README" or ".gitignore." Click **Create repository**.
2. GitHub shows you a page with the repository's web address, like `https://github.com/yourname/ist688-app1`. Copy it.
3. In Claude Desktop's **Code** tab, with `app1` open, paste this (with your real address):

   > Set up git in this folder, create a Python .gitignore that also excludes `.streamlit/secrets.toml`, make a first commit, and push to https://github.com/yourname/ist688-app1

Claude may need you to sign in to GitHub the first time.

- **If GitHub opens a browser or sign-in window:** follow it and sign in normally.
- **If instead the terminal asks for a username and password:** use your **GitHub username**, not your email. GitHub does **not** accept your account password for Git pushes. You need a **personal access token** instead. Go to https://github.com/settings/tokens, create a token with access to your repository, copy it, and paste the token when Git asks for your password.

4. Refresh your repository page on GitHub. You should see your files there.
5. Go to https://share.streamlit.io, sign in with GitHub, allow access, click **Create app**, pick `ist688-app1`, and make sure the main file is `app.py`.

The first deploy can take a few minutes. If it fails, click the logs on Community Cloud and paste them to Claude.

One last test — prove your development loop works

6. In Claude Code, open `app.py` again and change the title from **"Tip Calculator"** to **"My Tip Calculator"** (or another obvious title).
7. Save the file and check that the new title appears in the **local preview**.
8. Then ask Claude to **commit and push the change to GitHub**.
9. Refresh your `.streamlit.app` page. The new title should appear automatically — you do **not** need to create or deploy the app again.

Pass signal: your changed title appears on your public `.streamlit.app` link. **Post that link in the course channel** — that's how the checklist gets marked complete.

You have now completed the full workflow you'll use throughout the course: **open the code → edit → test locally → commit → push → see the live app update**.

Gate 9 — API keys (only when you need one)

Skip this until your project actually calls an API. Come back then.

1. Put the key in a file called `.streamlit/secrets.toml` inside `app1`. Claude can create it.
2. In `app.py`, read it with `st.secrets["YOUR_KEY_NAME"]`. **Never paste the key into `app.py`.**
3. For the live version, add the same key under your app's settings on share.streamlit.io.
4. Go to your repo on github.com and search for the key. It must not be there.

Pass signal: the live app works, and the key is nowhere in your GitHub repo.

A key that ends up in a public repo is a graded incident, not an oops. Bots find them in minutes.

Quick fixes

What you see	Try this first
Claude can't find `uv` or `git`	Quit Claude completely, then reopen it
`uv: command not found` immediately after installing	Close the Terminal/PowerShell window completely, open a brand-new one, and try `uv --version` again
No **Code** tab	Personal account, or Claude Code access not yet granted — see Gate 1
Preview blank and spinning	Missing `--server.headless true`
Git asks for a password	Use a GitHub personal access token, **not** your GitHub account password — see Gate 8

What you see	Try this first
“repository already exists”	Pick a different name
Anything else	Paste the exact error to Claude with this README attached

---

Notes for Claude

If a student has shared this file with you, these instructions are for you. Claude Code should also follow the project instructions in `CLAUDE.md`.

- The student is following this checklist. Walk them through it **one gate at a time**, and wait for the pass signal before moving on.
- Before diagnosing anything, ask which operating system they’re on and for the **exact error text**. Students will open with “it doesn’t work.” Get the text.
- Keep them on the tools in this document: **Claude Desktop’s Code tab, `uv`, Git, GitHub, Streamlit Community Cloud.** Do not suggest `pip`, `venv`, `conda`, `pipenv`, `poetry`, Anaconda, VS Code, Cursor, Codespaces, Docker, Heroku, or any other alternative, even if it would also work. Consistency across the class matters more than the best tool for one student.
- Do not tell them to install Python or Streamlit directly. `uv` installs both automatically on the first `uv run`.
- The fix for Claude not seeing a newly installed command is usually: **quit Claude Desktop completely and reopen it.** If uv was just installed and the shell itself says `command not found`, close the old terminal and open a brand-new one first.
- A missing Code tab has two causes: signed in on a personal account instead of SU SSO, or the separate Claude Code access request at getclaude.syr.edu hasn’t been submitted or approved yet. Check both. A common mistake is completing the Claude **Cowork** training instead of the Claude **Code** training — the cards look alike. Ask which one they did. SU access requests take roughly an hour to activate, sometimes longer — both the initial Claude Chat request and the Claude Code request after the training video. If a student just submitted either one, the answer is to wait, not to troubleshoot.
- Cowork is not required for this course and may not appear as a separate top-level tab in every Claude Desktop version.
- If the fix isn’t in this document and isn’t obvious, say so plainly and send them to the course channel with the exact error text. Don’t improvise a workaround.
- `pyproject.toml` and `requirements.txt` must stay in sync. If you add a dependency to one, add it to the other.